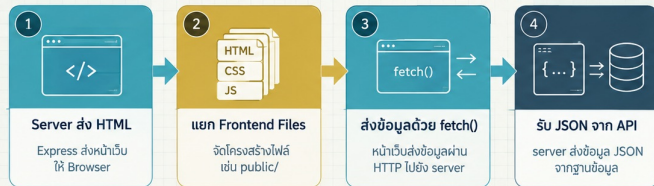


Database Integration using SQL II

คู่มือแบบ Hand-by-Hand: จาก Express Server สู่ fetch() + JSON
เรียนรู้การทำงานของ Express, static files, และ API แบบเป็นขั้นตอน

เส้นทางการเรียนรู้ (Roadmap)



วันนี้จะได้อะไร

- เข้าใจการทำงานของ Express เบื้องต้น
- เห็นโครงสร้างไฟล์ของเว็บจริง
- ส่งข้อมูลจากหน้าเว็บไปหา server
- แสดงผลโดยไม่ต้อง refresh หน้า

Step 1: เริ่มจาก Express Server แบบง่าย

ตั้งโปรเจกต์และเตรียมไฟล์แรก

เป้าหมาย

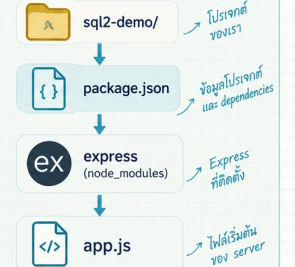
- เข้าใจว่า server ส่งหน้าเว็บได้
- รู้จัก Express เบื้องต้น
- เห็นภาพการทำงานของ route "/"

คำสั่งสร้างโปรเจกต์

```
Terminal
$ mkdir sql2-demo
$ cd sql2-demo
$ npm init -y
$ npm install express
```

จากนั้นสร้างไฟล์ `app.js`

โครงสร้างไฟล์ที่ได้



Step 2: เขียน app.js และเปิด server

ให้ Browser เรียก GET / แล้วรับ HTML กลับ

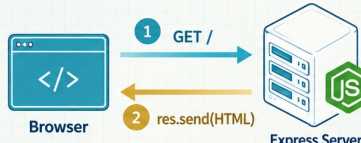
app.js

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("<h1>Hello Backend</h1>");
});

app.listen(3000, () => {
  console.log("Server started");
});
```

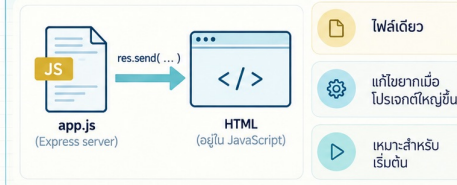
การทำงานของ Step 2



รันด้วยคำสั่ง: `node app.js`
เปิด: `http://localhost:3000`

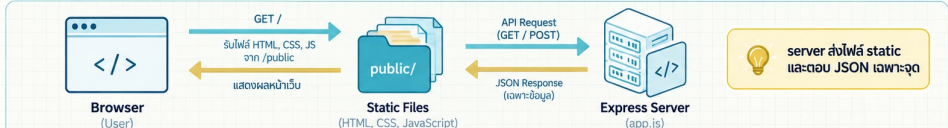
ก่อนหน้านี้

หน้าเว็บถูกฝังในโค้ดของ server



แบบที่เราจะใช้ต่อ

แยกไฟล์เป็นส่วนส่วนชัดเจน



MODERN WEB DEVELOPMENT | NODE.JS | EXPRESS | POSTGRESQL

Step 4: เขียน **app.js** สำหรับ static files และ API

รับ JSON จาก fetch() และส่งผลลัพธ์กลับ

```

1 const express = require("express");
2 const path = require("path");
3 const app = express();
4
5 app.use(express.json());
6 app.use(express.static(path.join(__dirname, "public")));
7
8 app.post("/calculate", (req, res) => {
9   const r = Number(req.body.radius);
10  const area = Math.PI * r * r;
11  res.json({ area: area });
12 });
13
14 app.listen(3000, () => {
15   console.log("Server started at http://localhost:3000");
16 });

```

คำอธิบายโค้ด

- express.json()**
อ่านข้อมูล JSON จาก request body
- express.static(...)**
เปิดให้ Browser โหลดไฟล์ใน public/
- POST /calculate**
รับ radius แล้วส่ง area กลับเป็น JSON

Frontend (Browser) → POST /calculate { "radius": 5 } → Express Server (app.js) → { "area": 78.54 }

WEB DEVELOPMENT FOR REAL SOLUTIONS | FROM FRONTEND TO DATABASE | 5

MODERN WEB DEVELOPMENT | NODE.JS | EXPRESS | POSTGRESQL

Step 5: สร้างหน้าเว็บด้วย **index.html** และ **style.css**

ทำหน้า input ให้พร้อมสำหรับเรียก API

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta charset="UTF-8">
5   <title>Circle</title>
6   <link rel="stylesheet" href="style.css">
7 </head>
8 <body>
9   <div class="box">
10    <h2>คำนวณพื้นที่วงกลม</h2>
11    <input type="number" id="radius" placeholder="ใส่รัศมี">
12    <button onclick="calculate()">คำนวณ</button>
13    <p id="result">ผลลัพธ์จะแสดงตรงนี้</p>
14  </div>
15  <script src="script.js"></script>
16 </body>
17 </html>

```

public/style.css

```

1 body {
2   font-family: Arial;
3   background: #f0f4f8;
4 }
5 .box {
6   background: white;
7   padding: 30px;
8   border-radius: 10px;
9   text-align: center;
10 }

```

ผลลัพธ์หน้าเว็บ (Preview)

คำนวณพื้นที่วงกลม

ใส่รัศมี: 5 คำนวณ

ผลลัพธ์จะแสดงตรงนี้

FROM FRONTEND TO DATABASE | 6

MODERN WEB DEVELOPMENT | NODE.JS | EXPRESS | POSTGRESQL

Step 6: เขียน **script.js** เพื่อส่งข้อมูลด้วย **fetch()**

ส่งค่า radius ไปหา server และรับผลลัพธ์กลับแบบไม่ต้อง refresh หน้า

```

1 function calculate() {
2   const r = document.getElementById("radius").value;
3
4   fetch("/calculate", {
5     method: "POST",
6     headers: {
7       "Content-Type": "application/json"
8     },
9     body: JSON.stringify({ radius: r })
10  })
11  .then(res => res.json())
12  .then(data => {
13    document.getElementById("result").innerText =
14      "พื้นที่ = " + data.area.toFixed(2);
15  });
16 }

```

- ผู้ใช้กรอกรัศมี ใส่ค่ารัศมีในหน้าเว็บ
- JavaScript เรียก fetch() ใช้ fetch('/calculate', { method: 'POST', ... })
- Browser ส่ง POST /calculate ส่งข้อมูล radius ในรูปแบบ JSON { "radius": 5 }
- Server คำนวณและส่ง JSON กลับ ค่าพื้นที่วงกลมจากรัศมี แล้วส่งผลลัพธ์เป็น JSON { "area": 78.54 }
- หน้าเว็บอัปเดตข้อความผลลัพธ์ แสดงผลลัพธ์ในหน้าเว็บ โดยไม่ต้อง refresh หน้า

⚡ AJAX style: หน้าไม่ refresh

FROM FRONTEND TO DATABASE | 7

MODERN WEB DEVELOPMENT | NODE.JS | EXPRESS | POSTGRESQL

Complete Flow + Key Takeaways

ภาพรวมการทำงานตั้งแต่หน้าเว็บจนถึง API

- Browser โหลดไฟล์ (index.html, style.css, script.js) จาก Express (static files)
- ผู้ใช้ใส่ค่า radius = 5 ใน input และกดปุ่มคำนวณ
- script.js เรียก fetch() ส่งข้อมูลไปที่ /calculate
- Express รับ JSON { "radius": 5 } และส่งข้อมูลกลับไปหน้าเว็บ (req.body)
- Server คำนวณ: area = Math.PI * r * r (r = 5)
- ส่งกลับผลลัพธ์ { "area": 78.54 } ส่ง JSON กลับไปยังหน้าเว็บ
- พื้นที่ = 78.54 หน้าเว็บแสดงผล

สรุปสิ่งที่ต้องจำ

- HTML** = โครงสร้าง กำหนดหน้าเว็บ เช่น หัวข้อ ปุ่ม และพื้นที่แสดงผล
- CSS** = ความสวยงาม จัดรูปแบบหน้าเว็บ ให้ดูดีและใช้งานง่าย
- JavaScript** = การโต้ตอบ + fetch() รับค่าจากผู้ใช้ เรียก API และแสดงผล
- Express** = server + API รับคำร้องขอ (Request) ประมวลผล และส่งคำตอบ (Response)

พร้อมต่อยอด

- เชื่อม SQLite หรือ PostgreSQL เพื่อเก็บข้อมูลอย่างถาวร
- บันทึกข้อมูลจากหน้าเว็บ เช่น บันทึกประวัติ และผลลัพธ์คำนวณได้
- ดึงข้อมูลกลับมาแสดงผล แสดงประวัติการคำนวณ หรือสถิติ

WEB DEVELOPMENT FOR REAL SOLUTIONS | 8

Step 7: เชื่อมต่อฐานข้อมูลด้วย SQLite

จากเว็บที่คำนวณอย่างเดียว สู่เว็บที่บันทึกข้อมูลได้

From
Frontend
to Database
Step by Step

1 ติดตั้ง package

```
npm install express sqlite3
```

2 app.js — เริ่มต้นฐานข้อมูล

```
const sqlite3 = require("sqlite3").verbose();
const db = new sqlite3.Database("circle.db");

db.run(`
  CREATE TABLE IF NOT EXISTS circles (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    radius REAL,
    area REAL
  )
`);
```

3 ภาพรวมการเชื่อมต่อฐานข้อมูล



4 โครงสร้างตาราง circles

ชื่อคอลัมน์	ชนิดข้อมูล	คำอธิบาย
id	INTEGER PRIMARY KEY AUTOINCREMENT	รหัสลำดับ (เพิ่มอัตโนมัติ)
radius	REAL	รัศมีของวงกลม
area	REAL	พื้นที่ของวงกลม

CREATE TABLE IF NOT EXISTS = สร้างตารางเฉพาะตอนที่ไม่มี

เมื่อรันครั้งแรก จะเกิดไฟล์ circle.db

Step 9: อ่านข้อมูลจากฐานข้อมูล

สร้าง route /history เพื่อดึงประวัติการคำนวณทั้งหมด

From
Frontend
to Database
Step by Step

GET /history

server.js

```
1 app.get("/history", (req, res) => {
2   db.all(
3     "SELECT * FROM circles ORDER BY id DESC",
4     [],
5     (err, rows) => {
6       if (err) {
7         return res.json([]);
8       }
9       res.json(rows);
10    }
11  });
```

การทำงานของ /history



ตัวอย่างข้อมูลที่ได

id	radius	area
3	10	314.16
2	5	78.54
1	3	28.27

ข้อมูลจะแสดงจากใหม่สุดไปเก่าสุด (id มากไปน้อย)

Key Takeaways

- db.all() = อ่านหลายแถว ใช้สำหรับดึงข้อมูลหลายรายการจากฐานข้อมูล
- ORDER BY id DESC = ใหม่อันดับแรก
- res.json(rows) = ส่งข้อมูลกลับเป็น JSON ส่งผลลัพธ์ที่ได้ออกไปยัง Frontend ในรูปแบบ JSON

Step 10: แสดง History บนหน้าเว็บ

ดึงข้อมูลจาก /history แล้วสร้างรายการบนหน้าเว็บ

public/script.js — loadHistory()

```
1 function loadHistory() {
2   fetch("/history")
3     .then(res => res.json())
4     .then(data => {
5       const list = document.getElementById("history");
6       list.innerHTML = "";
7
8       data.forEach(item => {
9         const li = document.createElement("li");
10        li.innerHTML =
11          "รัศมี = " + item.radius +
12          " | พื้นที่ = " + item.area.toFixed(2);
13        list.appendChild(li);
14      });
15    });
16 }
```

ประวัติการคำนวณ

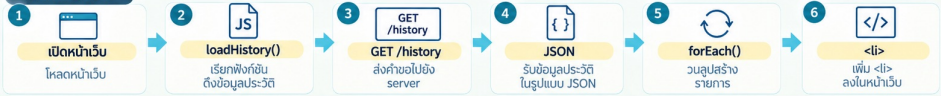
- รัศมี = 10 | พื้นที่ = 314.16
- รัศมี = 5 | พื้นที่ = 78.54
- รัศมี = 3 | พื้นที่ = 28.27

โค้ด HTML สำหรับรายการ

```
1 <h3>ประวัติการคำนวณ</h3>
2 <ul id="history"></ul>
```

หลังจากบันทึกใหม่ ให้เรียก loadHistory() อีกครั้ง

ลำดับการทำงาน



Step 11: ลบข้อมูลด้วย DELETE

ส่ง id ของแถวที่ต้องการลบจากหน้าเว็บไปยัง server

DELETE ต้องเลือกแถวด้วย
WHERE id = ?

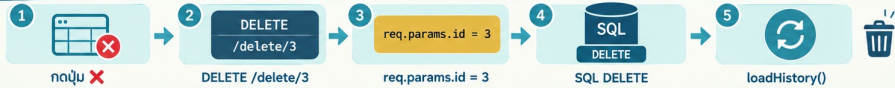
Backend — DELETE Route

```
app.delete("/delete/:id", (req, res) => {
  const id = req.params.id;
  db.run(
    "DELETE FROM circles WHERE id = ?",
    [id],
    function(err) {
      if (err) {
        return res.json({ success: false });
      }
      res.json({ success: true });
    }
  );
});
```

Frontend — deleteItem()

```
function deleteItem(id) {
  fetch("/delete/" + id, {
    method: "DELETE"
  })
  .then(res => res.json())
  .then(data => {
    if (data.success) {
      loadHistory();
    }
  });
}
```

การทำงาน



Step 12: แก้ไขข้อมูลด้วย PUT

รับรัศมีใหม่ คำนวณ area ใหม่ แล้ว UPDATE แถวเดิม

From
Frontend
to Database
Step by Step

Backend — PUT Route

app.js (Express)

```
app.put("/update/:id", (req, res) => {
  const id = req.params.id;
  const newRadius = Number(req.body.radius);
  const newArea = Math.PI * newRadius * newRadius;
  db.run(
    "UPDATE circles SET radius = ?, area = ? WHERE id = ?",
    [newRadius, newArea, id],
    function(err) {
      if (err) {
        return res.json({ success: false });
      }
      res.json({ success: true });
    }
  );
});
```

Frontend — editItem()

script.js

```
function editItem(id) {
  const newRadius = prompt("ใส่รัศมีใหม่");
  if (!newRadius) return;
  fetch("/update/" + id, {
    method: "PUT",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ radius: newRadius })
  })
  .then(res => res.json())
  .then(data => {
    if (data.success) {
      loadHistory();
    }
  });
}
```

Key Idea

เปลี่ยน radius →
ต้องคำนวณ area ใหม่ด้วย

ลำดับการทำงาน



Complete CRUD Flow

จาก Frontend → Express → SQL → Database

From
Frontend
to Database
Step by Step

Browser (Frontend)	Express (Backend)	SQL (Query)	SQLite Database (Data)
CREATE สร้างข้อมูล	POST /users	INSERT INTO users (...)	เพิ่มข้อมูล
READ อ่านข้อมูล	GET /users	SELECT * FROM users	อ่านข้อมูล
UPDATE แก้ไขข้อมูล	PUT /users/:id	UPDATE users SET ... WHERE id = ?	แก้ไขข้อมูล
DELETE ลบข้อมูล	DELETE /users/:id	DELETE FROM users WHERE id = ?	ลบข้อมูล

ภาพรวมการทำงาน (6 ขั้นตอน)



สิ่งที่ทำได้แล้ว

- ✓ คำนวณโดยไม่ refresh หน้า
ตอบสนองแบบทันที ด้วย JavaScript
และ fetch()
- ✓ บันทึกข้อมูลลงฐานข้อมูล
ส่งข้อมูลจาก Frontend ผ่าน Express
ไปยัง SQLite/PostgreSQL
- ✓ แสดงประวัติ
ดึงข้อมูลจากฐานข้อมูลและแสดงผล
บนหน้าเว็บ
- ✓ แก้ไขและลบข้อมูล
อัปเดตและลบข้อมูลได้ผ่าน API
จากหน้าเว็บ



นี่คือโครงสร้าง Mini Web App ที่มี Frontend + Backend + Database + API